

Radar Micro-Doppler Classification

A getting-started guide for the team

Lab project, Signals and Systems for Automated Driving, FH Dortmund. August 2026.

This document is a map, not an implementation guide. Its purpose is that all five of us start from the same picture of the project: what we are building, which hardware and software carries which role, where the relevant repositories and documentation live, how the work splits into streams, and which decisions are still open. Read it once, end to end, before the first team meeting. Follow the links only where your workstream needs depth. Nothing here contains code, and everything here should survive contact with reality for at least a few weeks.

1. What we are building

The project turns the radar sensor from the lab kit into a small classification system. The sensor observes a moving target, an embedded neural network decides whether that target is a pedestrian, a cyclist, or nothing of interest, and the result appears live on a display. Everything in this guide serves that single sentence.

The professor's slides frame the project around the RTB Radar-Button, a pedestrian crossing button that uses a 60 GHz radar with one transmit and one receive antenna to measure range and speed. Among the suggested research directions, our team has chosen micro-Doppler classification: distinguishing pedestrians, bicycles, scooters and similar road users from their radar signatures, with tinyML as the method. This is a realistic industrial question. A crossing button that knows whether a pedestrian or a cyclist is approaching can adapt its behaviour, and radar is attractive for this because it works in darkness, rain and fog, costs little, consumes little power, and, unlike a camera, records no personally identifiable images.

The physical effect we exploit is called micro-Doppler. A radar measures the velocity of a target through the Doppler shift of the reflected signal. A rigid object such as a car body produces essentially one velocity. A walking person does not: the torso moves at one speed while arms and legs swing forward and backward around it, so the reflected signal contains a whole spread of velocities that oscillates with the gait cycle. A cyclist adds the rotation of pedals and wheels, which produces a different pattern. Plotted over time, these velocity patterns form characteristic textures, and a small neural network can learn to tell them apart. The figure referenced on the professor's second slide shows exactly such [micro-Doppler images for a pedestrian, a bicycle, a motorbike and a car](#).

One honest limitation must be stated at the outset, because it shapes the whole scope. Our sensor has a single receive antenna, so it measures range and radial velocity but no angle. We cannot localise targets in two dimensions, we cannot separate two people standing side by side, and a target crossing perpendicular to the beam produces almost no bulk Doppler. Our system is therefore a single-target classifier that works best on targets approaching or departing, which happens to match the crossing-button use case well. Stating this clearly in the report and the final presentation is a strength, not a weakness.

2. The pipeline from sensor to display

The system is best understood as a chain of eight stages. The first five run on a PC during development. Only the last three run on the embedded hardware in the final demo.

Sensing. The XE125 evaluation board carries the XM125 module, which combines the Acconeer A121 radar sensor with a small STM32 host microcontroller. The A121 is a pulsed coherent radar: it emits very short 60 GHz pulses and measures the returning echo coherently, meaning with phase information, which is what makes velocity measurement possible. For our purposes the sensor is operated in the Sparse IQ service, which delivers raw complex samples per range point and per sweep. Everything else in the project is computed from this data.

Streaming. During development the XE125 is connected to a PC over USB and runs Acconeer's exploration server firmware. The PC side is the [Acconeer Exploration Tool](#), a Python application that configures the sensor, visualises live data, and records it. This is where every team member should start, because watching the live plots while someone walks in front of the sensor builds more intuition in ten minutes than any paper.

Recording and dataset building. The Exploration Tool saves recordings as HDF5 files that contain every frame together with the exact sensor configuration. Our dataset is simply a well organised folder of such files, one class per recording, following the naming protocol described in section 6.

Feature extraction. Raw IQ data is not fed to the network directly. A Fourier transform across consecutive sweeps converts the data into velocity spectra, and stacking those spectra over time yields the micro-Doppler spectrogram, an image-like array with time on one axis and velocity on the other. This representation is the standard one in the literature and has the pleasant property that the classification problem starts to look like small-image recognition.

Training. A compact convolutional network, typically a few tens of kilobytes of weights, is trained on the spectrograms using an ordinary PC. Whether this happens in a Python notebook or inside Edge Impulse is the subject of section 5.

Quantisation. The trained floating point model is converted to eight bit integers. This shrinks it by roughly a factor of four and makes it fast on a microcontroller, at a small and measurable cost in accuracy.

Embedded inference. In the final system the ESP32-S3 talks to the XE125 directly, computes the same feature extraction in C, and runs the quantised model with the TensorFlow Lite Micro runtime. It is worth underlining that the FFT and spectrogram code on the target is real signal processing work of our own, not something the ML runtime provides.

Display. The ESP32-S3-LCD-EV-Board shows the live class, a confidence value, and ideally the scrolling spectrogram itself, which makes for a far more convincing demo than a text label alone.

One process rule follows from this chain: we build a thin version of the whole pipeline early, with a trivial model and ugly plots, before we perfect any single stage. Integration is where lab projects lose weeks, and a dummy end-to-end system by mid-project is the insurance against that.

3. The hardware and what each board is for

The lab kit contains more boards than the final system needs. Each item has a defined role, and two of them are deliberately held in reserve.

Item	Role in the project
Acconeer XE125 (A121 radar)	The sensor. Streams Sparse IQ data to the PC during development and to the ESP32-S3 in the final system.
ESP32-S3-LCD-EV-Board	The target platform. Runs feature extraction, TensorFlow Lite Micro inference and the demo display.
SparkFun Thing Plus ESP32-S3	Development and field node. Useful as a portable data logger and as a second unit so that deployment work never blocks display work.
STM32 Nucleo-L476RG	Held in reserve. Backup host platform via the STM32 AI toolchain if the ESP32 path fails, otherwise unused.
USB logic analyzer, 24 MHz, 8 channel	Debugging the serial link between the XE125 and the ESP32-S3 when the two are first connected.

Two practical notes on the XE125. First, its onboard STM32 ships with detector firmware, and switching between detector firmware and the exploration server requires flashing the module over USB with STM32CubeProgrammer using the DFU and RESET buttons. The [SparkFun hookup guide](#) documents this procedure step by step for the identical SparkFun board, and the [product page](#) links schematics and datasheets. Second, the radar antenna is sensitive to nearby objects, so cables and mounting hardware must stay out of the area in front of the sensor, and the board should be mounted on a stable tripod or bracket from the very first recording session, because a wobbling sensor contaminates the Doppler data.

4. The toolchain and where to find it

Five software packages carry the whole project. Every team member should know what each one is for, even if only two people ever configure it.

[Acconeer Exploration Tool](#) is the PC application for live visualisation and data recording. It installs from PyPI as `acconeer-exptool` and its documentation lives at [docs.acconeer.com](#). The same documentation site explains the A121 sensor itself, the Sparse IQ service, and the ready-made [detectors](#) and [example applications](#), which are worth skimming to understand what Acconeer provides out of the box and where our own work begins.

[The Acconeer SDK](#) provides the C libraries and firmware images for the XM125, including the exploration server binary we flash for streaming. Downloading it requires a free developer account, which one team member should register early.

[The carpet detection example](#) on Acconeer's GitHub is our most important template, despite its unglamorous name. It demonstrates the complete machine learning workflow for the A121 in one repository: recorded data, a notebook that walks through feature extraction, model definition, training and evaluation, and a script that runs the resulting classifier in real time against the live sensor. Our manual pipeline is

structurally this repository with different data and different classes. Further examples are collected on the [Acconeer GitHub organisation page](#).

[esp-tflite-micro](#) is Espressif's packaging of the TensorFlow Lite Micro runtime as an ESP-IDF component, including the ESP-NN kernels that use the S3's vector instructions to accelerate inference. It is added to a project with a single dependency line. Working with it requires the [ESP-IDF development framework](#), whose getting-started guide covers installation and the basic build, flash and monitor cycle. Team members new to ESP-IDF should build the plain hello_world example before touching anything radar related.

[Edge Impulse](#) is a browser-based platform that covers data ingestion, feature extraction, training, quantisation and export in one place, with [documented support for ESP32 targets](#). Its role in our project is discussed in the next section. For completeness, ST's [X-CUBE-AI](#) is the equivalent deployment toolchain for the Nucleo board and is mentioned here only because it is our documented fallback.

5. Edge Impulse or a manual pipeline

This is the one genuine tooling decision the team has to make, and it deserves an honest comparison rather than a quick vote. The recommendation at the end is to use both, in sequence, with a fixed decision date.

The case for Edge Impulse is speed and integration. The platform takes uploaded sensor data, lets us design a processing and learning pipeline in the browser, trains and quantises the model, estimates on-device latency and memory for our exact target, and exports everything as a self-contained C++ library that drops into an ESP-IDF project. For a five-person team in which not everyone has machine learning experience, this removes an enormous amount of friction, and it would let us answer the single most important question of the project, namely whether our classes are separable at all with this sensor, within the first weeks. The free developer tier is sufficient for a project of our size.

The case against relying on it alone is threefold. First, control and understanding: the course is about signals and systems, and a pipeline in which the feature extraction is a configured block rather than code we wrote weakens both our learning and the story we tell in the final report. Radar spectrograms are also not a native Edge Impulse input type, so our data would enter the platform through a custom conversion step we must write anyway, which erodes part of the convenience. Second, flexibility: experimenting with radar-specific ideas, such as different sweep-to-frame arrangements or normalisation schemes, is easier in our own notebook than inside a web UI. Third, dependence: exported projects have historically lagged behind the newest ESP-IDF releases, and debugging inside a large generated codebase is harder than debugging code we assembled ourselves.

The manual pipeline inverts these properties. Following the structure of the carpet detection repository, we would write the feature extraction in Python ourselves, define a small Keras model, train it, convert it with the TensorFlow Lite converter, and integrate the resulting model file with esp-tflite-micro by hand. This maximises understanding, grading credit and flexibility, at the price of more work and more opportunities to get stuck, particularly in the quantisation step and in keeping the C feature extraction numerically identical to the Python version, which is a classic source of silent accuracy loss.

The recommendation is a two-track approach. In the first weeks, one or two members push a quick prototype through Edge Impulse using an initial small dataset, purely to test class separability and to obtain a latency and memory estimate on the S3. In parallel, the training workstream builds the manual notebook pipeline, which is the one we intend to ship and describe in the report. At a fixed date, roughly a third of the way into the project, we compare results and commit: if the manual pipeline is healthy we continue it, and if it is badly stuck the Edge Impulse export becomes the fallback rather than the plan. Under no circumstances do we run both to the end.

6. How we collect data

The dataset decides the outcome of this project more than any modelling choice. The protocol below makes labelling trivial, keeps the data legally clean, and is designed so that any team member can run a recording session alone.

The central rule is one class per recording. We do not record mixed street scenes and annotate them afterwards. Instead, each recording session captures a single known target type, so the label is simply part of the file name and no frame-level annotation is ever needed. A session consists of the sensor mounted on a tripod at roughly one metre height, matching the mounting height of a crossing button, pointed at an open area such as an empty parking area, a wide campus path, or a long corridor for the first indoor tests. One team member then passes through the beam repeatedly while another operates the Exploration Tool.

Within each class we deliberately vary the conditions: approaching, departing and diagonal trajectories, two or three distance bands within roughly one to six metres, and slow, normal and fast passes. File names encode everything, following a fixed pattern such as `ped_approach_3m_normal_p2_take5.h5`, which records class, geometry, speed, subject and take number. A shared logbook sheet notes date, location, weather and the sensor configuration used. This sounds bureaucratic and takes thirty seconds per recording, and it is what allows us to diagnose problems weeks later.

Three further rules matter. First, we record a proper background class from day one: the empty scene, swaying vegetation, a rolling trolley, an umbrella, rain if we are lucky enough to get it. A classifier that has never seen nothing will confidently hallucinate pedestrians. Second, our subjects are the team members themselves, with varied clothing, bags and bicycles, not passers-by. Recording uninvolved people, especially with a synchronised camera, raises data protection questions under the GDPR that we simply avoid by design. A short recording session in a genuinely public setting can serve as a final validation set at the end, with a phone video as ground truth and with care to record only consenting participants. Third, we freeze the sensor configuration early, within the first exploratory weeks, because recordings made with different sweep rates or range settings cannot be cleanly mixed into one training set. The configuration freeze is a milestone, not an afterthought.

As a working target, we aim for at least one hundred passes per class, from at least three different subjects, in at least two locations, with one location held back entirely as a test set. Segmentation inside each file is automated with a simple energy threshold that cuts out the windows where a target is actually present. If the collection plan feels tedious, it helps to remember that every published radar classification paper spends most of its length on exactly this.

7. Background reading

Nobody needs to read everything. The list is ordered so that the first three items give any team member enough shared vocabulary to follow every project discussion.

Begin with the [Acconeer documentation](#), specifically the introductory pages on the A121 sensor and the Sparse IQ service, which explain in accessible language what the sensor actually measures and what the terms sweep, frame, profile and PRF mean. These terms appear in every configuration file we will touch. Then look at the [micro-Doppler figure](#) cited on the professor's slide until the four textures make intuitive sense: the pedestrian's rhythmic gait pattern, the cyclist's flickering pedal returns, the smoother traces of the motorised targets. Third, read through the [carpet detection notebook](#) on GitHub without running anything, simply to see the shape of the workflow we are copying.

For deeper dives, the [detector documentation](#) shows what Acconeer's own presence and speed algorithms do, which is useful both for the automatic segmentation of our recordings and for understanding what we are adding beyond the vendor's offering. On the embedded side, the [esp-tflite-micro component page](#) and its hello_world example are the gentlest introduction to how a model actually executes on a microcontroller, including the tensor arena concept that will dominate our memory budget discussions. Members of the training workstream should additionally read any short tutorial on post-training int8 quantisation in TensorFlow Lite. The classic textbook on the physics, for those who want it, is V. C. Chen, The Micro-Doppler Effect in Radar, though it goes far beyond what the project needs.

8. How we divide the work

Five people, five workstreams, and every pair of neighbouring workstreams connected by one explicitly named hand-over artifact. The names are proposals for the team meeting, not assignments.

Sensor and acquisition. Owns the XE125: flashing, configuration, the tripod rig, and the connection to the PC and later to the ESP32-S3. Delivers a documented, frozen sensor configuration and a reliable recording setup. Hands over: the configuration file and a how-to-record instruction sheet.

Dataset. Owns the collection campaign, the naming convention, the logbook, the segmentation script and dataset quality. Runs the recording sessions with rotating helpers from the whole team, since data collection is the one task that parallelises perfectly. Hands over: versioned dataset releases with a documented train, validation and test split.

Features and model. Owns the notebook pipeline: spectrogram computation, model architecture, training, evaluation and quantisation, plus the early Edge Impulse prototype from section 5. Hands over: a quantised model file together with an exact written specification of the expected input, meaning shape, data type, scaling and normalisation.

Embedded deployment. Owns the ESP-IDF project on the ESP32-S3: the driver link to the sensor, the C implementation of the feature extraction, and inference with esp-tflite-micro. The critical duty is proving, with recorded test vectors, that the C features match the Python features bit for bit within tolerance. Hands over: a firmware that outputs class probabilities on a defined internal interface.

Display, integration and demo. Owns the user-visible result on the LCD board, the end-to-end steel thread from the first weeks onward, latency measurements, and the final demo and presentation materials. This workstream has the right to interrupt the others, because integration problems are everyone's problems.

9. Milestones

Six checkpoints, each with a demonstrable outcome. Dates belong to the team meeting; the order is fixed by the dependencies.

M0, environment: every member has the Exploration Tool running and has seen live radar data; ESP-IDF builds `hello_world` on at least two machines. M1, first data: the rig stands, the configuration is drafted, and a first mini-dataset of a few passes per class exists. M2, separability: the Edge Impulse prototype or a first notebook model shows clearly better than chance accuracy on the mini-dataset, and the sensor configuration is frozen. This is the go or no-go point for the class set. M3, dataset release one: the full collection campaign is complete to the targets of section 6, and the manual pipeline reaches a stated accuracy on the held-out location. M4, on target: the quantised model runs on the ESP32-S3 on live sensor data with measured latency, even with a rough display. M5, demo: the integrated system runs the live demo scenario reliably, and report and presentation are drafted. The steel thread rule from section 2 means a skeletal version of M4 exists long before M3 is finished.

10. Risks and open decisions

Four risks worth naming before they find us, and five decisions the team meeting must close. None of them is a reason to change direction; all of them are reasons to scope carefully.

The first risk is physics: crossing targets carry little radial velocity, so we scope the demo around approaching and departing targets and treat perpendicular motion as a documented limitation. The second is class overlap: a standing scooter rider is nearly a rigid body and may be indistinguishable from other smooth movers, so the scooter is a stretch class, and the core promise is pedestrian versus cyclist versus background. The third is domain shift: a model trained in one location can fail in another, which our two-location collection rule and held-out test location exist to expose early rather than late. The fourth is the classic schedule failure of lab projects, in which the model works offline in week ten and nothing runs on hardware until the final week; the steel thread and the M4-before-M3 overlap are the countermeasures.

The open decisions, with recommendations: the class set (recommend pedestrian, cyclist, background as core, scooter as stretch); Edge Impulse versus manual (recommend the two-track plan of section 5 with a decision at M2); where inference runs (recommend the ESP32-S3, with the XM125's own Cortex-M mentioned in the report as an outlook, since running on the module itself is the more elegant embedded story but a materially harder path); whether crossing trajectories are in scope (recommend out, revisited only if M3 accuracy leaves headroom); and the configuration freeze date (recommend coupling it to M2). Once these five are closed, everything else in this document is execution.

This guide is a living document. When reality disagrees with it, and it will, we update the document rather than quietly diverging from it.